

## Stochastic Sampling Algorithms

## Stochastic sampling algorithms

$p(\theta | d)$  is probability distribution

$$\langle x \rangle = \int x p(x) dx$$

$$\rightarrow \langle \theta \rangle = \int \theta p(\theta | d) d\theta$$

Very hard to calculate in practice  $\sim \mathcal{O}(N^3)$

$$\langle \theta \rangle = \frac{\sum \theta_i}{N} \quad (\text{Technically, this is a sample mean, not the true mean})$$

Procedure for generating these samples?

i.i.d. independent and identically distributed

We want a random number generator for  $p(\theta | d)$

independent: sample  $i+1$  doesn't depend on sample  $i$

`numpy.random.normal`  $\rightarrow$  makes i.i.d.

Samples  $\mathcal{N}(0, 1)$

" " `.uniform`  $\Rightarrow$  makes i.i.d.

Samples from  $U[0, 1]$

## Markovian Processes

Chain of events, next step only depends on your current state.

$$p(x_n | x_0, x_1, \dots, x_{n-1}) = p(x_n | x_{n-1})$$

## Markov Chain Monte Carlo (MCMC) :

Algorithm for generating approximately i.i.d samples (a Markov chain) from a target distribution

The Markov chain must satisfy the detailed balance condition (stationary)

$$\underbrace{p(x'|x)p(x)}_{\text{Prob. that } x' \text{ follows } x \text{ in the Markov chain. (We don't know this)}} = p(x|x')p(x') \quad (\text{Bates' theorem})$$

Prob. that  $x'$  follows  $x$  in the Markov chain.  
(We don't know this)

Replace  $p(x'|x)$  with a guess that we can calculate,  $q(x'|x)$  "Proposal distribution"

- ↳  $\mathcal{N}(x, \sigma^2)$
- ↳  $U[x_{\min}, x_{\max}]$

Our detail balance equation becomes an inequality:

$$q(x'|x)p(x) > q(x|x')p(x')$$

The chain moves from  $x \rightarrow x'$  too often  
and  $x' \rightarrow x$  too rarely

To recover detail balance, reweight by a transition probability  $\alpha \in (0, 1)$

$$q(x'|x)p(x)\alpha(x, x') = q(x|x')p(x')\alpha(x', x)$$

When  $x' \rightarrow x$  is too rare, we shouldn't make it worse!

$$\alpha(x', x) = 1$$

$$\alpha(x', x) = \frac{p(x') q(x|x')}{p(x) q(x'|x)} \rightarrow \text{Hastings ratio} \equiv H(x', x)$$

If the inequality was the other way, then  $H(x, x') > 1$ , but  $\alpha(x, x') \in [0, 1]$ .

$$\alpha(x, x') = \min[1, H(x, x')]$$

### Metropolis-Hastings Algorithm

Start at state  $x$

→ Propose a new state  $x'$  w/  $q(x'|x)$   
Accept it with probability  $\alpha(x, x')$   
if accept, state  $\rightarrow x'$   
if reject, state  $\rightarrow x$

Repeat until you converge!  
or you run out of time!

Now let's map it to our problem:

state  $\rightarrow$  parameters  $\theta$

target distribution  $\rightarrow$  posterior  $p(\theta|d)$

$$H(\theta, \theta') = \frac{p(\theta') p(d|\theta') q(\theta|\theta')}{p(\theta) p(d|\theta) q(\theta'|\theta)}$$

- Pro: - Convergence is mathematically guaranteed (eventually)  
 - Simple to implement

- Cons: - inherently serial  $\rightarrow$  slow  
 - inefficient if  $q$  mismatches posterior  
 - does not directly estimate evidence  
 - Needs fine tuning to perform well!  
 More "art" than science.

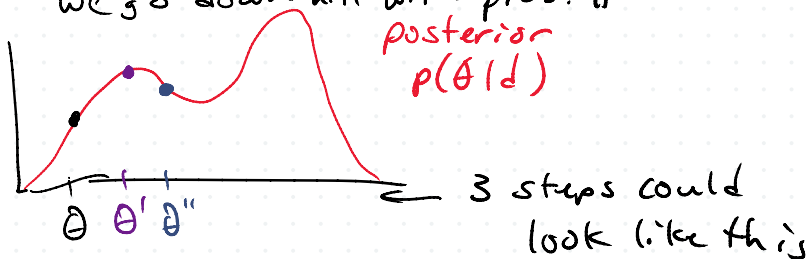
## Special cases

Uniform priors, symmetric proposal

$$\alpha(\theta, \theta') = \frac{p(d|\theta')}{p(d|\theta)} \quad (\text{just the likelihood ratio!})$$

So if  $p(d|\theta') > p(d|\theta)$ , we Accept,  
 we always go uphill.

If  $p(d|\theta') < p(d|\theta)$ , we might accept,  
 we go downhill with prob.  $\alpha$



Going downhill is very important, allows us to jump to other peaks!

2/ Proposal distribution matches target distribution

$$\#(\theta, \theta') = \frac{p(\theta' | \mathcal{D}) q(\theta | \theta')}{p(\theta | \mathcal{D}) q(\theta' | \theta)} = \frac{q(\theta' | \theta)}{p(\theta' | \mathcal{D})}$$

} = 1 everything gets accepted!  
perfectly efficient!

[Note: super efficient proposals also exist, basically, they fill in gaps faster than random chance would]

Designing good proposals is hard work, but is worth it!

3/ Hammer at the true value

$$q(\theta' | \theta) = \delta(\theta - \theta^T)$$

$$\#(\theta, \theta^T) = \frac{p(\theta^T | \mathcal{D})}{p(\theta | \mathcal{D})} \cdot \frac{\delta(\theta - \theta^T)}{\delta(\theta)} = 0.??$$

The proposal is like a teenager, the harder you push, the harder it pushes back!

proposals must be:

- 1) continuous
- 2) normalizable
- 3) span the full parameter space

4/ The full thing:

$$H(\theta, \theta') = \underbrace{\frac{p(d|\theta') p(\theta')}{p(d|\theta) p(\theta)}} \cdot \underbrace{\frac{q(\theta|\theta')}{q(\theta'|\theta)}}$$

Always go uphill, but  
try downhill as well

Counterbalance  
how hard you're  
pushing

## lecture notes 22 (continued)

Really great MCMC reference! (long HW, implement your own MCMC by following these notes):

<https://arxiv.org/abs/1710.06068>

### MCMC chain intuition building

We want our MCMC chains to look like **fuzzy caterpillars**, not **wiggly worms**. In other words, basically you want them to look like white noise! (Small autocorrelation length)

Signs of bad chains include lots of rejected proposals (streaks), or too small or too big jumps (wormy chains).

Some bad chain examples:

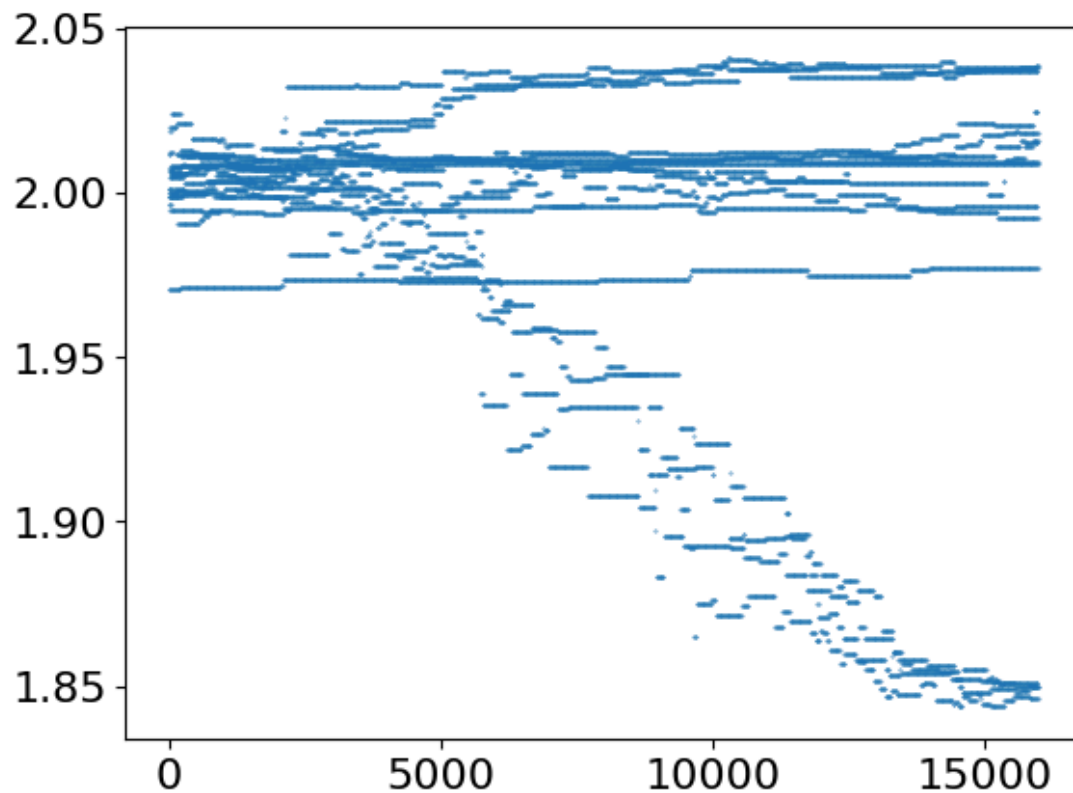


Figure 1: 46584740d2ee04390d0e5fc9bd167b70.png

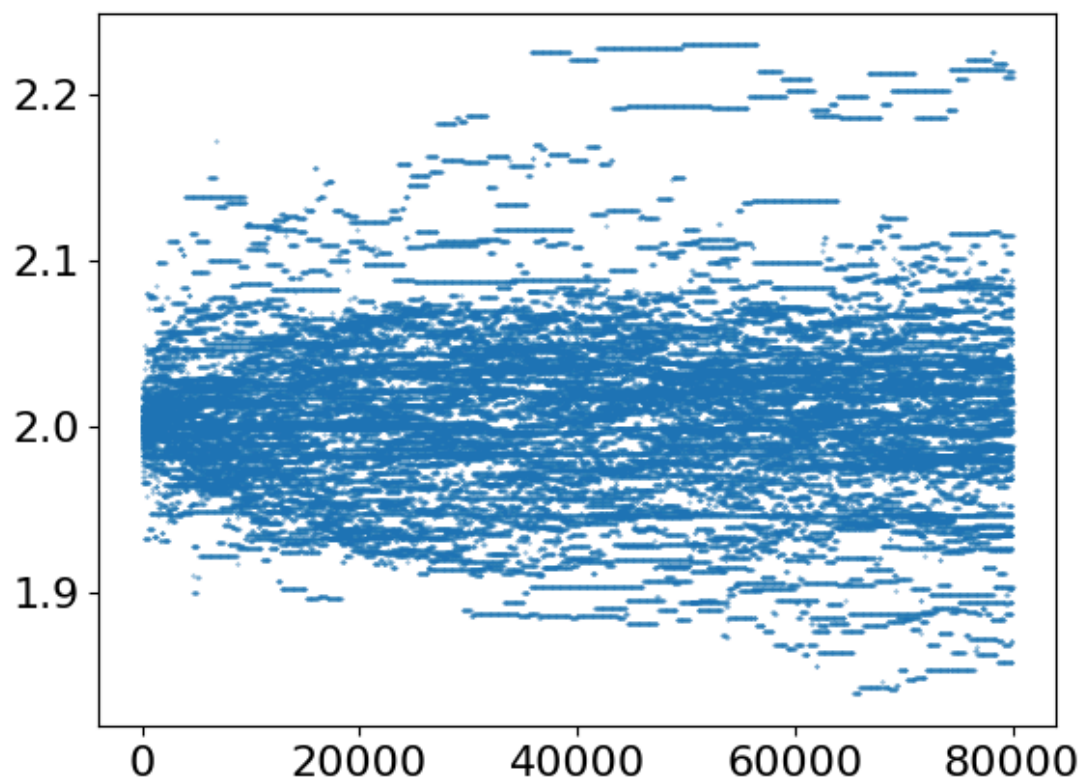


Figure 2: fe3f7c8b8a9f6b99281eed6e0927135.png

Some good chain examples:

